

Chapitre 1 : La complexité des algorithmes

27/08/2025

1 Introduction

Comment peut-on savoir si un algorithme est plus efficace qu'un autre, ou même savoir s'il est performant ? On a vu en S1 qu'on pouvait mesurer le temps dans différents cas, mais ceci dépend tellement des conditions (machine, système d'exploitation, langage, mémoire, etc.) et du nombre de cas analysés, cette méthode est souvent très chronophage.

Si un algorithme met un temps t_0 sur une machine, il mettra environ deux fois moins de temps sur une machine deux fois plus rapide. De même, selon le langage (la quantité de mémoire, etc.) qu'on utilisera, le même algorithme ne mettra pas le même temps. De même, si on augmente le nombre de données, le temps va augmenter, par forcément de façon proportionnelle.

Pour ne pas dépendre des conditions, on va devoir analyser la complexité de l'algorithme. Ainsi, on pourra conclure que cette complexité est convenable, ou la comparer avec d'autres algorithmes.

L'analyse de la complexité d'un algorithme consiste en l'étude formelle de la quantité de ressources nécessaire à l'exécution de cet algorithme. Celle-ci ne doit pas être confondue avec la théorie de la complexité, qui elle étudie la difficulté intrinsèque des problèmes, et ne se focalise pas sur un algorithme en particulier.

2 Histoire

Pendant les années 1950, les scientifiques ont voulu énoncer formellement et rigoureusement ce qu'est la complexité d'un algorithme, ils se sont rendu compte que la comparaison des algorithmes entre eux était nécessaire et que les outils pour le faire à l'époque étaient primitifs. Dans la préhistoire de l'informatique, la mesure publiée était souvent dépendante du processeur utilisé, des temps d'accès à la mémoire vive et de masse, du langage de programmation et du compilateur utilisé.

Une approche indépendante des facteurs matériels était donc nécessaire pour évaluer l'efficacité des algorithmes. Donald Knuth fut un des premiers à l'appliquer systématiquement, il complétait cette analyse de considérations propres à la théorie de l'information.

3 Différentes métriques

3.1 La complexité temporelle

Quand on compare différents algorithmes, souvent on prend en compte le facteur temps : combien de temps prend l'algorithme pour résoudre le problème.

On parle de complexité temporelle.

3.2 La complexité spatiale

Un autre point à connaître concerne le stockage des données. Un algorithme va stocker des informations dans la mémoire de l'ordinateur, or celle-ci n'est pas infinie. Ainsi un algorithme peut vite occuper tout l'espace libre d'un ordinateur, et le faire ralentir voir le faire planter.

On parle ici de complexité spatiale (en espace).

3.3 D'autres métriques

D'autres mesures existent :

- nombre de communications (bande passante)
- montée en charge : l'augmentation du nombre d'utilisateurs peut impliquer des points sensibles et critiques de l'architecture (goulot d'étranglement).
- stress : activité maximale attendue aux heures de pointes, remplissage de la mémoire, des temps de réponses.
- robustesse : activité maximale sur une longue durée, on mesure le temps sans dégradation des performances.
- résilience : activité en simulant des pannes, d'incident, de piratage ou d'augmentation des opérations. La résilience implique la mise en place de mesures de "sauvegarde", de protection des données (en cas de piratage, de vol ou de catastrophe), mais aussi de mettre en place des mesures préventives, comme la sécurité physique, la maintenance et l'administration des SI, etc.

4 Grand O de Landau

4.1 Introduction

On utilise une notation standard qui s'appelle grand O et qui permet de mesurer la performance d'un algorithme.

Si la complexité temporelle d'un algorithme est $O(f(n))$, ça signifie que le temps réellement mis sera $\lambda \times f(n)$, où λ dépend de la machine, du langage et d'autres paramètres. Si on double le nombre de données alors le temps réel sera $\lambda \times f(2n)$ dans les mêmes conditions précédentes.

Si la complexité spatiale d'un algorithme est $O(f(n))$, ça signifie que l'espace réellement utilisé sera $\lambda \times f(n)$, où λ dépend de la machine, du langage et d'autres paramètres.

Nous allons donner les fonctions couramment utilisées en complexité temporelle, il suffit d'adapter pour la complexité spatiale.

4.2 Échelle de comparaison

Voici une liste de catégories de fonctions couramment utilisées en complexité. n est la taille des données et c est une constante réelle arbitraire. Lorsque c est une constante supérieure à 1, les fonctions apparaissent dans cette liste par ordre croissant de grandeur.

notation	grandeur au plus
$O(1)$	module majoré par une constante
$O(\log(n))$	logarithmique
$O(\log^c(n))$	(polylogarithmique si c est entier positif)
$O(n)$	linéaire
$O(n \cdot \log(n))$	parfois appelée "linéarithmique"
$O(n \cdot \log^c(n))$	parfois appelé "quasi linéaire"
$O(n^2)$	quadratique
$O(n^c)$	(polynomiale si c est un entier positif)
$O(c^n)$	(exponentielle si c est positif, parfois "géométrique")
$O(n!)$	factorielle

4.3 $O(1)$: complexité constante

Ceci représente les algorithmes qui ont un temps d'exécution constant, quel que soit la taille de l'entrée. Par exemple, on a l'accès à un élément dans un tableau.

4.4 $O(\log(n))$: complexité logarithmique

Ceci représente les algorithmes dont le temps d'exécution augmente logarithmiquement avec la taille de l'entrée.

Par exemple, on a la recherche par dichotomie.

- Le logarithme naturel (ou népérien), souvent noté $\ln$, est fondamental en analyse mathématique, on a les propriétés suivantes :
 - $\ln(\exp(x)) = x$,

— $\exp(\ln(x)) = x$ (pour $x > 0$).

- Le logarithme décimal $\log_{10}$ (noté aussi $\log$), qui utilise la base 10, reste le plus communément utilisé pour les calculs dans le domaine technologique (calcul du pH, des décibels, etc.).

On a les propriétés suivantes :

- $\log_{10}(10^x) = x$,
- $10^{\log_{10}(x)} = x$ (pour $x > 0$),
- $\log_{10}(x) = \frac{\ln(x)}{\ln(10)}$.

Pour $x > 0$, si $n = \text{Ent}(\log_{10}(x))$, alors $x = y \cdot 10^n$ où $1 \leq y < 10$.

Donc $\log_{10}$ permet de connaître l'ordre de grandeur en écriture décimal.

- Le logarithme binaire $\log_2$ (noté aussi $\log$ en complexité des algorithmes), qui utilise la base 2, est très utilisé en informatique (mais aussi en musique).

On a les propriétés suivantes :

- $\log_2(2^x) = x$,
- $2^{\log_2(x)} = x$ (pour $x > 0$),
- $\log_2(x) = \frac{\ln(x)}{\ln(2)}$.

Pour $x > 0$, si $n = \text{Ent}(\log_2(x))$, alors $x = y \cdot 2^n$ où $1 \leq y < 2$.

En représentation virgule flottante binaire n est l'ordre de grandeur et y une significande.

Donc $\log_2$ permet de connaître l'ordre de grandeur en écriture binaire.

On remarque que $\log_2(x) = \log_{10}(x) \times \frac{\ln(10)}{\ln(2)} = \frac{\log_{10}(x)}{\log_{10}(2)}$, donc $O(\log_2(x)) = O(\log_{10}(x))$, et on notera simplement $O(\log(n))$, sans s'occuper de la base utilisée.

Rappel de propriétés remarquables des logarithmes (où $b > 0$, $x > 0$ et $y > 0$) :

- $\log_b(1) = 0$
- $\log_b(b) = 1$
- $\log_b(x \times y) = \log_b(x) + \log_b(y)$
- $\log_b\left(\frac{x}{y}\right) = \log_b(x) - \log_b(y)$
- $\log_b(x^p) = p \times \log_b(x)$

Par exemple, on a la recherche dichotomique.

4.5 $O(n)$: complexité linéaire

Ceci représente les algorithmes dont le temps d'exécution est linéairement proportionnel à la taille de l'entrée.

Par exemple, on a le parcours de tous les éléments d'un tableau.

4.6 $O(n \log(n))$: complexité linéarithmique

Ceci représente les algorithmes dont le temps d'exécution est proportionnel $n \log(n)$, où n est la taille de l'entrée.

Par exemple, on a le tri rapide.

4.7 $O(n^2)$: complexité quadratique

Ceci représente les algorithmes dont le temps d'exécution est proportionnel au carré de la taille de l'entrée.

Par exemple, on a le tri par sélection.

4.8 $O(2^n)$: complexité exponentielle

Ceci représente les algorithmes dont le temps d'exécution double à chaque fois que la taille de l'entrée augmente d'une unité.

Par exemple, les algorithmes de brute-force sont souvent dans cette classe.

4.9 Comparaison

n	1	$\log(n)$	n	$n \log(n)$	n^2	2^n
1	1	0	1	0	1	2
2	1	0,3	2	0,6	4	4
4	1	0,6	4	2,4	16	16
8	1	0,9	8	7,2	64	256
16	1	1,2	16	19,3	256	65 536
32	1	1,5	32	48,2	1 024	$4\,294\,967\,296 = 4,3 \cdot 10^9$
64	1	1,8	64	115,6	4 096	$1,8 \cdot 10^{19}$
128	1	2,1	128	269,7	16 384	$3,4 \cdot 10^{38}$
256	1	2,4	256	616,5	65 536	$1,2 \cdot 10^{77}$
512	1	2,7	512	1 387,1	262 144	$1,3 \cdot 10^{154}$
1024	1	3,0	1 024	3 082,5	1 048 576	$1,8 \cdot 10^{308}$

5 Différentes approches de la complexité

5.1 La complexité dans le pire des cas

La complexité dans le pire des cas, ou complexité dans le cas le plus défavorable, mesure la complexité d'un algorithme dans le pire des cas d'exécution possibles. Il s'agit par conséquent d'une borne supérieure des ressources requises par l'algorithme.

5.2 La complexité dans le meilleur des cas

La complexité dans le meilleur des cas correspond à la complexité d'un algorithme dans le cas d'exécution le plus favorable possible. Il s'agit par conséquent d'une borne inférieure des ressources requises par l'algorithme.

Elle est parfois nettement inférieure aux complexités en moyenne ou au pire des cas, et peut donc permettre de favoriser le choix d'un algorithme si l'on sait que les entrées à traiter ont en pratique de bonnes chances de correspondre à des cas optimaux.

5.3 La complexité en moyenne des algorithmes

La complexité en moyenne d'un algorithme est la quantité d'une ressource donnée, utilisée par l'algorithme lors de son exécution pour traiter une entrée tirée selon une distribution donnée. Il s'agit par conséquent d'une moyenne de la complexité, pondérée entre les différentes entrées possibles selon la distribution choisie. Le plus souvent, on ne précise pas la distribution et on utilise implicitement une distribution

uniforme (une loi uniforme discrète dans le cas discret), c'est-à-dire que toutes les entrées sont considérées comme équiprobables.

Certains problèmes ont une complexité dans le pire des cas très élevée, mais les entrées qui correspondent à ces cas ne se produisent que très rarement en pratique, de sorte qu'à part de ces cas extrêmes, la complexité en moyenne est une mesure plus utile de la performance de l'algorithme.

5.4 La complexité amortie

L'analyse amortie consiste essentiellement à majorer le coût cumulé d'une suite d'opérations pour attribuer à chaque opération la moyenne de cette majoration, en prenant en compte le fait que les cas chers surviennent rarement et isolément et compensent les cas bon marché. Pour être utilisable, cette analyse suppose que l'on est capable de borner la fréquence des cas les plus coûteux. L'analyse amortie se place dans le cas le plus défavorable et garantit la performance moyenne de chaque opération dans ce cas. À partir de l'analyse amortie on peut concevoir des structures de données efficaces.

Exemple

On a un tableau où soit on peut insérer 1 élément (coût 1), soit on génère un tableau λ fois plus grand où on recopie les n éléments du tableau précédent, puis on insère l'élément (coût total : $n+1$). Choisir de façon adéquate le coefficient d'accroissement λ , et calculer le coût pour la moyenne amortie

5.5 L'analyse lisse d'algorithme

Elle a été inventée dans les années 2000 par Daniel Spielman et Shang-Hua Teng.

L'introduction de cette nouvelle mesure de complexité a été motivée, d'une part par l'existence de certains algorithmes fonctionnant très bien en pratique mais dont l'analyse dans le pire des cas ne montrait pas l'efficacité, et d'autre part par la difficulté de l'analyse en moyenne en général.

Le principe de l'analyse lisse est de mesurer les performances de l'algorithme sur les pires cas, mais avec une légère perturbation des instances. On calcule donc une espérance de performance.

Plus précisément, on ajoute un bruit gaussien aux entrées, c'est-à-dire une variation aléatoire des données qui suit une loi normale. La complexité mesurée dépend alors de deux paramètres : la taille de l'entrée mais aussi l'écart-type de la gaussienne utilisée.

Si l'analyse lisse annonce pour un algorithme de bonnes performances parmi l'espace des valeurs centrées autour d'une donnée en ajoutant du bruit (gaussien) à cette donnée, cela signifie que, dans la pratique, il devrait y avoir aussi de bonnes performances pour cet algorithme dans tous les cas.

6 Exercices

Exercice n° 1 Rechercher la complexité spatiale des algorithmes suivants :

- 1) Comparaison de deux entiers.
- 2) Stockage d'un tableau de n éléments.
- 3) Stockage d'une matrice de $n \times n$ éléments.
- 4) Stockage d'une matrice de $n \times n$ éléments où la proportion de valeurs non nulles est p .

Exercice n° 2 Rechercher la complexité temporelle des algorithmes suivants :

- 1) L'accès à un élément dans un tableau.
- 2) Parcourir tous les éléments d'un tableau.
- 3) Le produit (naïf) de 2 matrices de $n \times n$ éléments.
- 4) Pour prouver naïvement qu'un entier est premier, on peut le diviser successivement par tous les entiers compris entre 2 et $\sqrt{n}$.

Exercice n° 3 Rechercher la complexité temporelle au pire et en moyenne

On arrive à une porte avec un code de n chiffres décimaux. On veut l'ouvrir, on utilise les algorithmes suivants :

- 1) On essaie tous les codes possibles les uns après les autres.
Donner l'algorithme (en pseudo-code) pour $n = 3$
- 2) Comme dans les films (ou séries), quand on cherche le code, on trouve les chiffres les uns après les autres.
(*Ceci vient que sur les anciens coffres-forts, quand on tournait la molette on pouvait entendre un "clic" quand on avait le bon chiffre*)
Donner l'algorithme (en pseudo-code) pour $n = 3$
- 3) On vient avec un merlin ou une masse, et on casse la porte.
(*Une entreprise de sécurité informatique avait demandé à un auditeur de prouver que les serveurs étaient inviolables, un merlin a prouvé le contraire*)

Exercice n° 4 Rechercher les complexités temporelles, spatiales, dans le pire des cas, en moyenne, et dans le meilleur des cas de l'algorithme suivant :

On prend un tableau, on vérifie si les éléments sont ordonnés et s'ils ne le sont pas, on mélange aléatoirement les éléments, puis on répète l'opération.

Comment se nomme ce tri ? (*tri utilisé par des étudiants dans un projet tutoré!*)

Exercice n° 5 Recherche dans un dictionnaire

Un dictionnaire est une liste de mots triés de façon lexicographique. Quelle est la complexité temporelle (dans le pire des cas, dans le meilleur des cas, en moyenne) des algorithmes de recherche suivants :

- 1) Parcourir les mots dans l'ordre jusqu'à trouver le mot cherché.
- 2) Prendre le mot central du dictionnaire (position $\text{Ent}\left(\frac{n}{2}\right)$), selon le mot cherché, on recommence cette méthode dans la 1ère moitié ou la seconde moitié du dictionnaire, jusqu'à qu'on trouve le mot cherché. (*nom de l'algorithme ?*)
- 3) Le hachage d'un mot est calculé à l'aide d'une fonction de hachage qui à chaque mot associe un entier (la probabilité que 2 mots différents donnent le même hachage, on dit dans ce cas qu'il y a collision, est quasiment nulle). La complexité du calcul du hachage est $O(1)$.

Les valeurs de hachage sont généralement utilisées pour être les indices d'une table (via un modulo) de taille raisonnable appelée table de hachage. Cette table de hachage est composée de listes de mots (en cas de collision des indices). On suppose que le choix de la fonction de hachage et du modulo, fait qu'on a un tableau de n listes (vides ou contenant peu de mots).

Quelle est la complexité de la recherche par hachage ?

Exercice n° 6 Sujet de DS

```

algorithme Somme(n : entier) : entier
variable
  ind : entier
  som : entier
début
  som ← 0
  pour ind de 1 à n faire som += ind fpr
  retourne som
fin

```

- 1) Que calcule cet algorithme Somme ?
- 2) Calculer la complexité de cet algorithme somme.
- 3) Donner un autre algorithme sommeOpt de complexité constante qui donne le même résultat.

Exercice n° 7 Sujet de DS

```

algorithme pwr(x : réel, n : entier) : réel    // ne gère que n >= 0
début
  si n=0 alors retourne 1 fsi
  si n=1 alors retourne x fsi
  retourne x * pwr(x,n-1)
fin

```

- 1) Que calcule cet algorithme pwr ?
- 2) Calculer la complexité de cet algorithme pwr.
- 3) Donner un autre algorithme pwrOpt basé sur le principe que : $x^{2p} = (x^p)^2$ et $x^{2p+1} = (x^p)^2 \times x$
- 4) Quels est le nombre d'opérations qu'il a fallu pour : $n = 2$, $n = 3$, $n = 4$ et $n = 5$?
- 5) Calculer la complexité de cet algorithme pwrOpt.